Docker OffGas

Documentazione tecnica e operativa

Progetto OffGas

Aprile 2026

Indice

1. Scopo del documento **2**

2. Ruolo di Docker nel sistema OffGas **2**

3. Approccio additivo del setup Docker **2**

4. Componenti eseguiti tramite Docker **3**
4.1 Servizio `mosquitto` . . . 3
4.2 Servizio `nodered` . . . 3
4.3 Perché il bridge resta fuori da Docker . . . 3

5. Volumi, mount e persistenza **4**
5.1 Runtime Node-RED persistente . . . 4
5.2 Dashboard montata come contenuto statico . . . 4
5.3 Dataset montati per il backend . . . 4
5.4 Configurazione di Mosquitto . . . 5

6. Integrazione con l'architettura applicativa attuale **5**
6.1 Catena dei dati . . . 5
6.2 Cosa gira dentro Docker e cosa no . . . 5
6.3 Effetto sul backend Node-RED . . . 5

7. Script operativi del repository **6**
7.1 Avvio dello stack . . . 6
7.2 Arresto dello stack . . . 6
7.3 Rebuild della dashboard . . . 6

8. Procedura operativa consigliata **6**
8.1 Primo avvio della parte software . . . 6
8.2 Uso con prototipo reale . . . 7
8.3 Aggiornamento della dashboard . . . 7

9. URL operativi e diagnostica **7**
9.1 Editor Node-RED . . . 7
9.2 Dashboard web . . . 7
9.3 Endpoint diagnostici utili . . . 7

10. Osservazioni operative importanti **8**
10.1 Discrepanza sul broker MQTT del bridge . . . 8
10.2 Struttura prevista del runtime Docker . . . 8
10.3 Docker non sostituisce la logica del progetto . . . 8

11. Vantaggi e limiti della soluzione adottata **8**
11.1 Vantaggi principali 8
11.2 Limiti intenzionali 9

12. Sintesi finale **9**

1. Scopo del documento

Questo documento descrive il ruolo del **setup Docker** nel progetto OffGas nella sua forma attuale.

L'obiettivo è spiegare:

- perché Docker è stato introdotto nel progetto;
- quali componenti del sistema vengono eseguiti nei container e quali invece restano fuori;
- come è organizzato il file `docker-compose.yml`;
- in che modo vengono montati dashboard, dataset e runtime Node-RED;
- come si avvia, si arresta e si aggiorna l'ambiente;
- quali sono i vantaggi e i limiti intenzionali di questa soluzione.

Il documento non tratta Docker come semplice strumento di packaging, ma come il livello che rende **riproducibile e condivisibile** la parte software del progetto OffGas senza spostare dentro i container la componente hardware reale.

2. Ruolo di Docker nel sistema OffGas

Nel progetto OffGas Docker viene usato per standardizzare l'ambiente di esecuzione della parte software che non dipende direttamente dall'hardware locale.

L'idea di fondo è separare chiaramente due aree:

1. **livello hardware locale**, che deve restare sulla macchina collegata al prototipo reale;
2. **livello software condivisibile**, che può essere eseguito in modo identico su macchine diverse.

In pratica, Docker viene usato per eseguire in modo coerente:

- il broker **Mosquitto**;
- **Node-RED** con il flow cloud del progetto;
- il serving statico della dashboard compilata.

Non viene invece containerizzato il **bridge Python**, perché è il componente che deve parlare con il modulo **HC-05** e quindi con la seriale Bluetooth del prototipo reale **G1**.

Docker ha quindi un ruolo architetturale preciso: non sostituisce il bridge e non tocca il firmware Arduino, ma fornisce un runtime stabile per la parte di orchestrazione e presentazione del sistema.

3. Approccio additivo del setup Docker

La configurazione Docker del progetto è stata pensata come **additiva**.

Questo significa che il setup non richiede di ristrutturare le cartelle storiche del repository e non impone di spostare la logica esistente in una nuova gerarchia. L'obiettivo è innestare il runtime containerizzato sopra il progetto già esistente, lasciando intatti i componenti principali.

In questa impostazione:

- `Bridge/` continua a contenere il bridge Python eseguito localmente;
- `offgas_dashboard_linked/` continua a essere la cartella sorgente della dashboard;
- `dataset_other_garage/` continua a essere la cartella modificabile dei dataset CSV;
- i file specifici del runtime Docker vengono collocati nell'area `docker/`, come previsto dalla configurazione.

Questo approccio è utile perché riduce il rischio di rompere il prototipo già funzionante e consente al team di introdurre Docker senza dover cambiare la logica hardware o il codice del bridge.

4. Componenti eseguiti tramite Docker

Il file `docker-compose.yml` definisce due servizi principali.

4.1 Servizio `mosquitto`

Il servizio `mosquitto` usa l'immagine:

```
eclipse-mosquitto:2
```

Il suo compito è fornire il broker MQTT su cui si appoggiano sia il bridge sia Node-RED.

La configurazione osservabile nel compose prevede:

- container name `offgas-mosquitto`;
- restart policy `unless-stopped`;
- pubblicazione della porta `1883:1883`;
- mount del file di configurazione `docker/mosquitto/mosquitto.conf` in sola lettura.

Dal punto di vista operativo, questo servizio svolge una funzione centrale: rende disponibile un broker MQTT uguale per tutti i membri del team, evitando installazioni locali manuali di Mosquitto.

4.2 Servizio `nodered`

Il servizio `nodered` rappresenta il backend applicativo del progetto.

Il compose lo costruisce tramite:

```
build:
  context: .
  dockerfile: docker/node-red/Dockerfile
```

Questo implica che il runtime di Node-RED non viene preso da un'immagine generica preconfezionata, ma da una build controllata dal progetto, pensata per includere le dipendenze e la configurazione necessarie al flow OffGas.

Il servizio espone:

- container name `offgas-nodered`;
- restart policy `unless-stopped`;
- dipendenza da `mosquitto`;
- variabili di ambiente `TZ=Europe/Rome` e `DATASET_DIR=/data/offgas-datasets`;
- pubblicazione della porta `1880:1880`.

Node-RED, in questa architettura, non è solo un editor di flow: è il vero backend del progetto. Nel sistema attuale, infatti, è il componente che gestisce:

- telemetria di **G1**;
- dataset delle unità simulate;
- calcolo di `others_mean`;
- calcolo della soglia dinamica;
- prediction e anomaly detection;
- endpoint HTTP per la dashboard;
- logica di controllo manuale e automatico.

4.3 Perché il bridge resta fuori da Docker

Il bridge Python comunica con l'hardware reale tramite Bluetooth seriale. Questo crea dipendenze molto più strette dal sistema operativo host rispetto ai componenti software puri.

Containerizzare il bridge significherebbe dover gestire almeno tre problemi aggiuntivi:

- mapping delle porte seriali o COM;
- permessi di accesso al dispositivo Bluetooth;
- dipendenze specifiche della macchina su cui è associato l'HC-05.

Questa complessità non porta un vantaggio reale, perché il bridge non deve essere condiviso come ambiente generico: deve semplicemente funzionare bene sulla macchina fisicamente collegata al prototipo.

Per questo motivo il progetto adotta una soluzione più pulita:

- **Docker** gestisce il livello cloud e di orchestrazione;
- il **bridge** resta locale e usa MQTT come punto di raccordo con i container.

5. Volumi, mount e persistenza

Una parte fondamentale del funzionamento di Docker in OffGas è la mappatura dei volumi tra host e container.

5.1 Runtime Node-RED persistente

Il compose monta:

```
./docker/nodered-data:/data
```

Questo significa che la cartella `/data` interna al container Node-RED viene persistita sul filesystem del progetto.

Dal punto di vista pratico, qui vengono mantenuti:

- flow e configurazioni del runtime;
- eventuali dipendenze installate nel contesto Node-RED;
- file pubblici serviti da Node-RED;
- stato persistente necessario a riaprire l'ambiente nelle stesse condizioni.

Il vantaggio è che il container può essere ricreato senza perdere automaticamente la configurazione del progetto.

5.2 Dashboard montata come contenuto statico

Il compose monta inoltre:

```
./offgas_dashboard_linked/dist:/data/public/offgas-dashboard:ro
```

Questo punto è importante: la dashboard non viene copiata manualmente dentro Node-RED a ogni aggiornamento. Viene invece compilata nella propria cartella `dist` e poi montata in sola lettura come contenuto statico del runtime Node-RED.

Di conseguenza:

- il codice sorgente della dashboard resta separato;
- la build finale viene servita direttamente da Node-RED;
- non serve un web server frontend separato;
- non serve un passaggio manuale di sincronizzazione dopo ogni build.

5.3 Dataset montati per il backend

Il compose prevede anche il mount:

```
./dataset_other_garage:/data/offgas-datasets:ro
```

Questo conferma che, nell'architettura Docker del progetto, i dataset dei garage simulati vengono messi a disposizione direttamente del runtime Node-RED.

Questo dettaglio è coerente con la revisione corrente dell'architettura: **dataset e threshold logic appartengono al backend Node-RED**, non al bridge e non al frontend.

Il mount in sola lettura è inoltre una scelta sensata, perché i CSV devono essere letti dal sistema ma non modificati accidentalmente dai container durante l'esercizio.

5.4 Configurazione di Mosquitto

Il servizio MQTT monta:

`./docker/mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro`

In questo modo la configurazione del broker è versionabile e controllabile dal progetto invece di dipendere da un'installazione locale esterna.

6. Integrazione con l'architettura applicativa attuale

Docker non cambia il significato dei componenti di OffGas, ma cambia **dove** girano alcuni di essi.

6.1 Catena dei dati

Con il setup Docker attivo, la catena operativa diventa:

Arduino -> HC-05 -> Bridge locale -> MQTT su host:1883 -> Mosquitto in Docker -> Node-RED in Docker -> Dashboard servita da Node-RED

Nel verso opposto, i comandi seguono questa strada:

Dashboard -> API HTTP di Node-RED -> MQTT -> Bridge locale -> HC-05 -> Arduino -> Ventola

Questa organizzazione mantiene separati in modo netto:

- il percorso fisico del prototipo reale;
- il percorso logico e applicativo del backend;
- il livello di interfaccia utente.

6.2 Cosa gira dentro Docker e cosa no

Nel setup corrente:

- dentro Docker girano **Mosquitto** e **Node-RED**;
- la dashboard compilata viene servita da **Node-RED**;
- fuori Docker restano **Arduino**, **HC-05** e **Bridge Python**.

Questo significa che il progetto non è "tutto in Docker". Ed è una scelta corretta: una containerizzazione completa sembrerebbe più elegante solo in astratto, ma complicherebbe la parte davvero sensibile, cioè il collegamento al prototipo reale.

6.3 Effetto sul backend Node-RED

Nel sistema attuale il runtime Docker non ospita un Node-RED vuoto, ma il backend operativo del progetto. Questo ha tre conseguenze pratiche:

1. la logica di soglia e dataset vive nel container Node-RED;
2. la dashboard legge i dati da endpoint HTTP esposti dal container;
3. il bridge locale deve limitarsi a pubblicare telemetria minima e ricevere comandi.

In altre parole, Docker non serve solo per "avere Node-RED acceso", ma per distribuire un backend già configurato e coerente con il resto del progetto.

7. Script operativi del repository

Il repository include una piccola serie di script che evitano al team di dover ricordare ogni volta i comandi Docker completi.

7.1 Avvio dello stack

Per l'avvio sono previsti:

- `scripts\up.bat` su Windows;
- `./scripts/up.sh` su macOS/Linux.

Entrambi eseguono sostanzialmente:

```
docker compose up --build -d
```

Questa scelta implica che ogni avvio ricostruisce, se necessario, il servizio Node-RED a partire dal Dockerfile del progetto, invece di limitarsi a riutilizzare uno stato potenzialmente incoerente.

7.2 Arresto dello stack

Per l'arresto sono previsti:

- `scripts\down.bat` su Windows;
- `./scripts/down.sh` su macOS/Linux.

Gli script eseguono:

```
docker compose down
```

Questo arresta i container del progetto senza richiedere che l'utente ricordi nomi di container o altri dettagli operativi.

7.3 Rebuild della dashboard

Quando il codice frontend cambia, la cartella `dist` deve essere rigenerata prima di riavviare o ricaricare l'ambiente Docker.

Per questo esistono:

- `scripts\rebuild-dashboard.bat`;
- `./scripts/rebuild-dashboard.sh`.

Gli script eseguono, nella cartella `offgas_dashboard_linked/`:

```
npm install
npm run build
```

Il risultato finale è una nuova build statica in `dist`, che viene poi servita automaticamente da Node-RED grazie al mount descritto in precedenza.

8. Procedura operativa consigliata

Dal punto di vista pratico, l'uso del progetto con Docker può essere descritto come una sequenza operativa semplice.

8.1 Primo avvio della parte software

1. Verificare che Docker sia disponibile sulla macchina.
2. Dalla root del progetto, eseguire lo script `up` appropriato al sistema operativo.
3. Attendere la creazione o l'avvio dei container `offgas-mosquitto` e `offgas-nodered`.
4. Verificare che Node-RED risponda sulla porta `1880`.

A questo punto la parte software condivisa del progetto è attiva, anche se il prototipo reale potrebbe non essere ancora collegato.

8.2 Uso con prototipo reale

Se si vuole lavorare con il prototipo reale **G1**, è necessario un passaggio ulteriore: avviare il bridge Python sulla macchina che vede l'HC-05.

La sequenza corretta è quindi:

1. avvio dello stack Docker;
2. verifica del broker MQTT e di Node-RED;
3. avvio del bridge locale;
4. verifica dell'arrivo della telemetria reale in Node-RED e nella dashboard.

Senza il bridge locale, Docker avvia correttamente il backend, ma non può acquisire i dati del prototipo fisico.

8.3 Aggiornamento della dashboard

Se si modifica il frontend:

1. ricompilare la dashboard con lo script `rebuild-dashboard`;
2. riavviare o ricaricare l'ambiente se necessario;
3. aprire di nuovo la pagina servita da Node-RED.

Il punto chiave è che il container non compila la dashboard automaticamente dal sorgente: si aspetta una build già pronta nella cartella `dist`.

9. URL operativi e diagnostica

La documentazione Docker del progetto individua due URL principali.

9.1 Editor Node-RED

`http://127.0.0.1:1880/admin/#flow/120ca2f2695d22bc`

Questo indirizzo serve ad accedere al canvas del flow e alle configurazioni Node-RED.

9.2 Dashboard web

`http://localhost:1880/offgas-dashboard/`

Questo è l'URL della dashboard servita dal runtime Node-RED.

9.3 Endpoint diagnostici utili

Dal backend servito in Docker è possibile interrogare anche endpoint HTTP di utilità, ad esempio:

```
http://localhost:1880/api/health
http://localhost:1880/api/state
http://localhost:1880/api/datasets
```

Questi endpoint sono utili per distinguere problemi diversi:

- se `api/health` risponde ma non arrivano valori reali, il problema è spesso nel bridge o nel collegamento Bluetooth;
- se Node-RED non risponde affatto, il problema è più probabilmente nello stack Docker;
- se la dashboard si apre ma non riflette il dataset atteso, il controllo va fatto su mount dei CSV e stato interno di Node-RED.

10. Osservazioni operative importanti

Analizzando i file del progetto emerge anche qualche dettaglio che merita attenzione.

10.1 Discrepanza sul broker MQTT del bridge

Nel materiale disponibile compaiono due indicazioni diverse per la configurazione MQTT del bridge:

- `README_DOCKER.md` suggerisce che il bridge locale possa continuare a usare `127.0.0.1:1883`, perché Mosquitto pubblica la porta sul sistema host;
- `Bridge/bridge.py`, nella configurazione mostrata, imposta invece `MQTT_BROKER = "host.docker.internal"`.

Queste due indicazioni non sono equivalenti in ogni contesto.

Se il bridge gira davvero **sulla macchina host**, la scelta più lineare è in genere `127.0.0.1`, perché il broker Docker è già esposto sulla porta dell'host. L'uso di `host.docker.internal` ha senso solo in scenari specifici e non andrebbe considerato automaticamente corretto in qualunque avvio locale.

Questo non invalida il setup Docker, ma è un punto da tenere presente nella documentazione operativa del progetto.

10.2 Struttura prevista del runtime Docker

Il compose fa esplicito riferimento a risorse sotto `docker/`, ad esempio:

- `docker/node-red/Dockerfile`
- `docker/nodered-data/`
- `docker/mosquitto/mosquitto.conf`

Questa è la struttura prevista dal runtime containerizzato. Di conseguenza, il setup Docker del progetto non dipende solo dal file `docker-compose.yml`, ma anche da questa area di supporto che contiene build, configurazioni e dati persistenti di esecuzione.

10.3 Docker non sostituisce la logica del progetto

Un errore concettuale possibile sarebbe pensare che Docker "gestisca" il comportamento di OffGas. In realtà Docker gestisce solo il contenitore di esecuzione.

La logica vera del sistema continua a stare in:

- **Node-RED** per soglia, dataset, prediction e API;
- **Bridge** per interfaccia Bluetooth/MQTT verso l'hardware;
- **Arduino** per acquisizione e attuazione fisica.

Docker rende questi componenti più facili da distribuire, ma non ne cambia le responsabilità.

11. Vantaggi e limiti della soluzione adottata

11.1 Vantaggi principali

La soluzione Docker adottata nel progetto offre diversi vantaggi concreti:

- riduce le differenze di ambiente tra macchine diverse;
- centralizza la configurazione di Mosquitto e Node-RED;
- rende riproducibile il backend del progetto;
- semplifica l'onboarding di altri membri del team;
- mantiene separata la parte hardware locale, che è la meno portabile.

Dal punto di vista progettuale, questa è una scelta più robusta di una containerizzazione totale forzata: si containerizza ciò che è davvero condivisibile e si lascia locale ciò che è intrinsecamente legato alla macchina.

11.2 Limiti intenzionali

La soluzione presenta anche alcuni limiti intenzionali, che però sono coerenti con gli obiettivi del progetto:

- il bridge va comunque avviato separatamente;
- il prototipo reale non è utilizzabile se il collegamento HC-05 non è configurato sull'host;
- la dashboard deve essere ricompilata quando cambia il frontend;
- il runtime Docker dipende dalla disponibilità delle cartelle montate correttamente dal repository.

Questi limiti non sono difetti accidentali, ma il prezzo pagato per mantenere stabile la parte hardware e condivisibile la parte software.

12. Sintesi finale

Il setup Docker di OffGas è stato progettato per rendere **riproducibile, portabile e semplice da avviare** il livello software del progetto senza forzare dentro i container la componente hardware reale.

In forma sintetica, Docker nel progetto:

- esegue **Mosquitto** e **Node-RED**;
- serve la dashboard compilata tramite static hosting di Node-RED;
- monta i dataset CSV per la logica delle unità simulate;
- mantiene persistente il runtime di Node-RED;
- lascia il bridge Python fuori dai container per preservare il collegamento Bluetooth col prototipo.

Questa architettura è coerente con la forma attuale di OffGas, in cui la logica di soglia, dataset e stato applicativo è centralizzata in **Node-RED**, mentre il bridge resta un gateway locale verso l'hardware reale.